作业 2

彭俊杰　智装备 241　2024306100324

1、说明下列英文缩写的含义：ARM、AHB、AHB-Lite、APB、PPB、DAP（JTAG、SWD）、MPU、FPU、NVIC、ISR、Bitband、SCS、SysTick、MSP、PC、SP、GPIO、TIM、UART、CANBUS。

缩写	英文名称	含义
ARM	Advanced RISC Machines	英国 ARM 公司及其处理器架构的名称。
AHB	Advanced High-performance Bus	AMBA 体系中的高性能总线，连接内核、存储器和高速外设。
AHB-Lite	Advanced High-performance Bus Lite	只支持单个主设备的 AHB 简化版本，去掉了总线仲裁。
APB	Advanced Peripheral Bus	面向低速低带宽外设的简单总线，经桥接入 AHB。
PPB	Private Peripheral Bus	内核私有的外设总线，连接 NVIC、SysTick 和调试部件。
DAP	Debug Access Port	调试访问端口，调试器经 JTAG 或 SWD 访问芯片内部资源。
JTAG	Joint Test Action Group	IEEE 1149.1 边界扫描标准接口，也用于芯片调试。
SWD	Serial Wire Debug	串行线调试接口，仅需 SWDIO、SWCLK 两根信号线。
MPU	Memory Protection Unit	存储器保护单元，把地址空间划分成区域并规定各自的访问权限。
FPU	Floating-Point Unit	浮点运算单元。Cortex-M4F 中的 FPU 仅支持单精度。
NVIC	Nested Vectored Interrupt Controller	嵌套向量中断控制器，负责中断的使能、挂起、优先级与嵌套管理。
ISR	Interrupt Service Routine	中断服务程序，处理器响应中断后跳转执行的例程。
Bitband	Bit-banding	位带机制：别名区的一个 32 位字对应位带区的一个位，读写别名即完成一次原子的位置位或清零。
SCS	System Control Space	系统控制空间，位于 PPB 区，集中存放 SCB、NVIC、SysTick 的寄存器。
SysTick	System Tick Timer	内核自带的 24 位递减定时器，常用作延时基准和 RTOS 时基。
MSP	Main Stack Pointer	主栈指针。复位后默认使用，处理异常时固定使用。
PC	Program Counter	程序计数器（R15），存放即将取回的指令地址。
SP	Stack Pointer	栈指针，即寄存器 R13。线程模式下可以选用 MSP 或进程栈指针 PSP。
GPIO	General-Purpose Input/Output	通用输入输出端口，引脚可逐个配置为输入或输出。
TIM	Timer	定时器外设，除定时计数外还可实现输入捕获与 PWM 输出。
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器。STM32 的 USART 另支持同步模式。
CANBUS	Controller Area Network Bus	控制器局域网总线，多节点共享介质，以逐位仲裁避免发送冲突。

2、简要画图说明 ARM Cortex-M4 对存储器映射如何划分地址区域的，（自学）标出 GPIO、SysTick、CAN 这些外设寄存器所在的地址在哪个区域？

Cortex-M4 的 32 位地址空间为 0x00000000～0xFFFFFFFF，共 4 GB，分区见图 1。

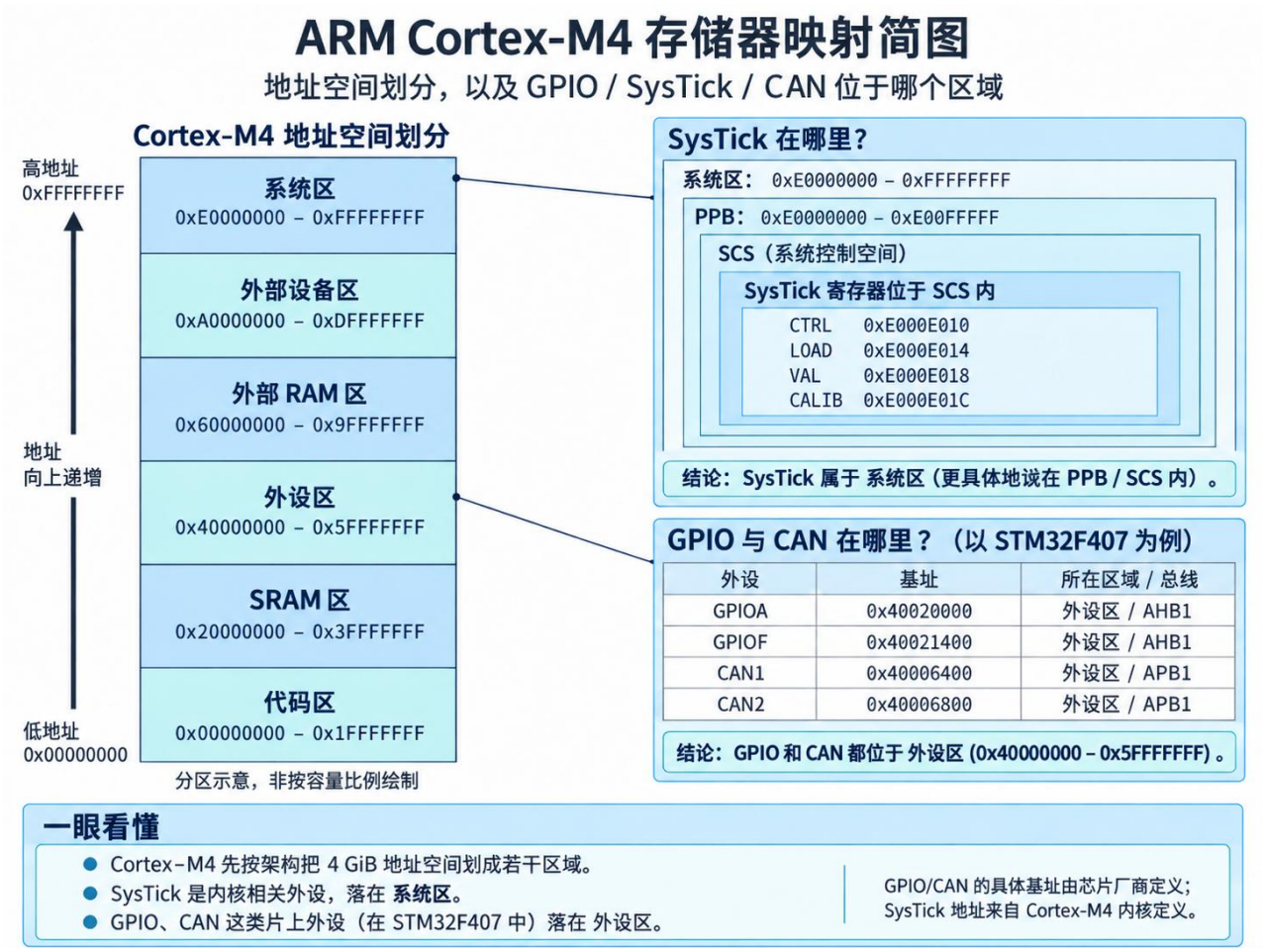


图 1 Cortex-M4 统一地址空间

对象	寄存器地址或基址	所属区域
GPIOA	0x40020000	外设区 → AHB1
GPIOF	0x40021400	外设区 → AHB1
CAN1 / CAN2	0x40006400 / 0x40006800	外设区 → APB1
SysTick	0xE000E010～0xE000E01C	系统区 → PPB → SCS

STM32F407ZGT6 的 1 MB Flash 位于 0x08000000～0x080FFFFF；SRAM1 与 SRAM2 合计 128 KB，位于 0x20000000～0x2001FFFF；64 KB CCM 数据 RAM 位于 0x10000000～0x1000FFFF，仅供 CPU 通过 D-bus 访问。

3、画简图说明有关中断的概念（罗列关键词），并说明 ARM Cortex-M4 微处理器异常/中断向量表的作用是什么？

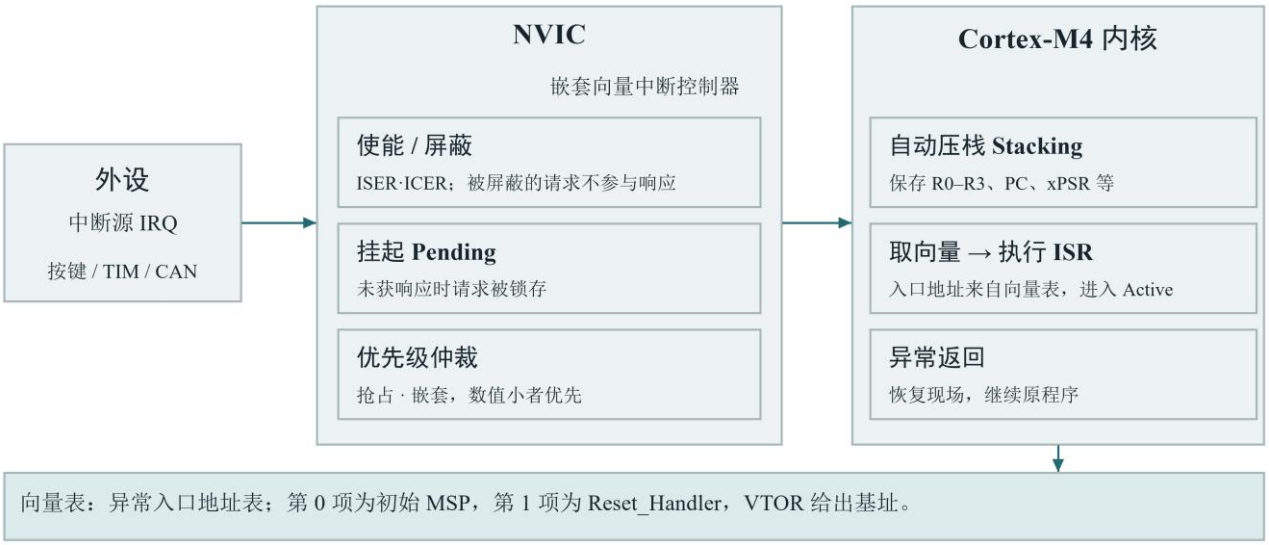


图 2 中断概念简图

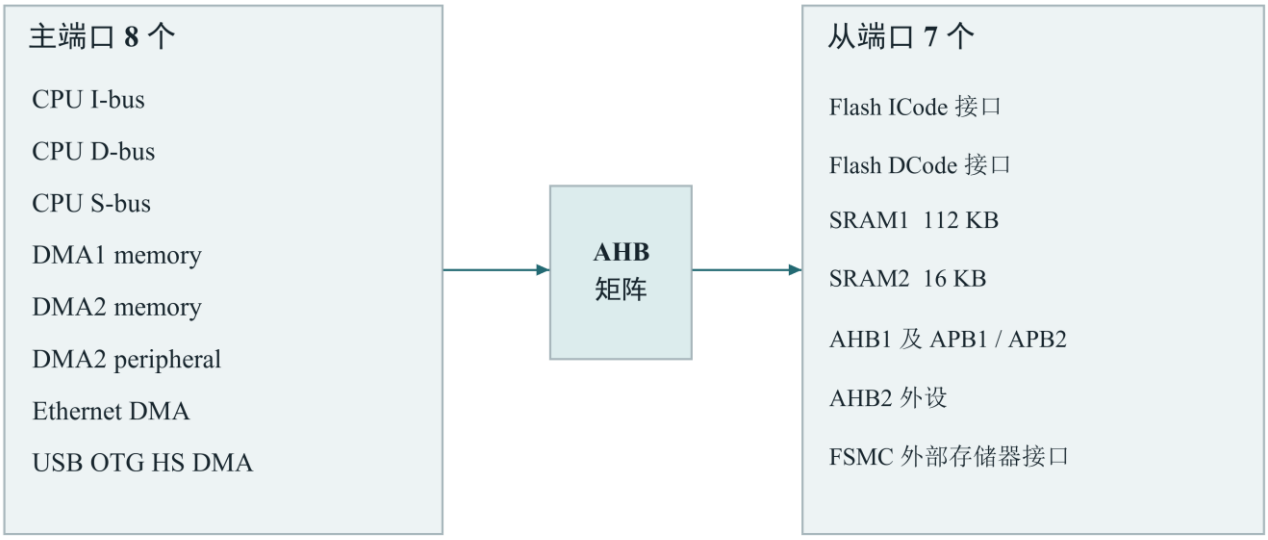
向量表的作用

向量表保存异常处理函数的入口地址，每项占 4 字节。第 0 项保存初始 MSP，第 1 项保存复位入口，其余为异常入口或保留项。外部中断 IRQn 的表项偏移为 $4 \times (16 + \mathrm{IRQn})$。

表项	偏移	内容
0	0x00	初始主栈指针 __initial_sp
1	0x04	Reset_Handler
2 / 3	0x08 / 0x0C	NMI_Handler / HardFault_Handler
11 / 14	0x2C / 0x38	SVC_Handler / PendSV_Handler
15	0x3C	SysTick_Handler
16 + IRQn	0x40 起	外部中断入口，例如 WWDG_IRQHandler

4、列写 STM32F40x 系列微控制器内部结构中片上外设主从总线有哪些，并说明 GPIO、TIMER2（或简写 TIM2）、USART1 模块分别挂在哪条总线上？

STM32F407 通过 32 位多层 AHB 总线矩阵连接处理器、DMA、存储器和外设。参考手册 RM0090 列出 8 个主端口和 7 个从端口，见图 3。



CCM 64 KB：CPU D-bus 专用访问；DMA1 外设端口直接连接 APB1。

图 3 STM32F407 总线矩阵及端口

外设连接

模块	挂接总线	基址与用途
GPIO	AHB1	GPIOA 基址 0x40020000；通用输入输出
TIM2	APB1	0x40000000；32 位通用定时器
USART1	APB2	0x40011000；同步或异步串行通信

AHB1 还连接 RCC、DMA 等控制器，并经总线桥接出 APB1 与 APB2；使用 GPIO 前需在 RCC 中打开对应的 AHB1 端口时钟。

5、STM32F40x 系列微控制器的高速和低速时钟源指的是哪些？如何得到系统时钟 SYSCLK？AHB1、APB1、APB2 总线的时钟最高频率各是多少？

类别	时钟源	频率与用途
高速内部	HSI	16 MHz RC；复位后的系统时钟来源
高速外部	HSE	4～26 MHz 晶体；本例采用 8 MHz
低速内部	LSI	标称约 32 kHz RC；可供独立看门狗、RTC 使用
低速外部	LSE	32.768 kHz 晶体；常用作 RTC 时钟
倍频输出	主 PLL	输入选择 HSI 或 HSE；经 M、N、P 得到 PLLCLK

SYSCLK 可以直接选择 HSI、HSE，或选择主 PLL 输出。采用 PLL 时，SYSCLK = f 输入 ÷ M × N ÷ P。例如 8 MHz ÷ 8 × 336 ÷ 2 = 168 MHz，配置示例如图 4 所示。

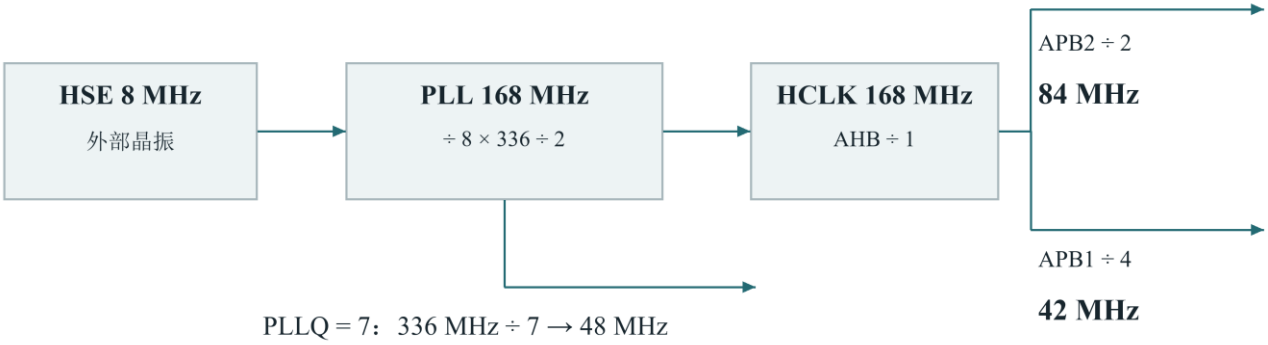


图 4 168 MHz 配置示例

时钟	最高频率	示例配置
SYSCLK	168 MHz	PLL 输出 168 MHz
AHB1 / HCLK	168 MHz	SYSCLK ÷ 1
APB1 / PCLK1	42 MHz	HCLK ÷ 4
APB2 / PCLK2	84 MHz	HCLK ÷ 2

APB 分频系数大于 1 时，定时器时钟为 2 × PCLK，本例中 APB1、APB2 定时器分别为 84 MHz、168 MHz。

6、在 STM32F407ZGT6 芯片引脚上找到复位脚，说明其名称的含义，并画出典型的复位电路。

STM32F407ZGT6 采用 LQFP144 封装，复位引脚为第 25 脚 NRST。N 表示低电平有效，RST 表示复位；将 NRST 拉低即可复位芯片。

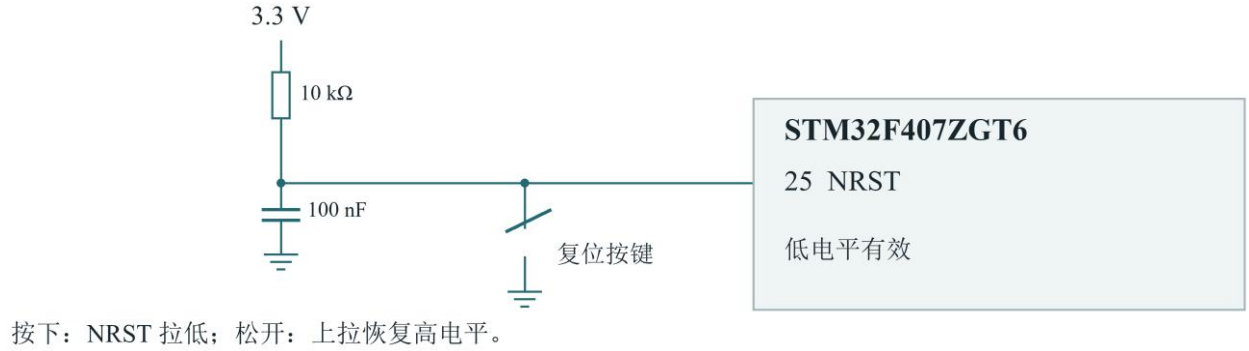


图 5 NRST 典型复位电路

按下按键，NRST 接地，芯片进入复位；松开后恢复高电平，处理器读取初始栈指针和复位入口，重新执行程序。

元件或信号	作用
10 kΩ 上拉电阻	让复位脚在按键松开时保持高电平。
100 nF 电容	与上拉网络配合抑制复位脚上的短暂干扰。
常开按键	闭合时将 NRST 接地，实现手动复位。
调试器 NRST	连接同一复位网络，允许调试器控制复位。

7、请查找我们实验所用两种开发板（407 通用、潘多拉物联网）的主芯片 STM32F407ZGT6、STM32L496 的 PDF 资料，简单描述一下其特征。

STM32F407ZGT6 与 STM32L496VET6 均采用 Cortex-M4 内核，带单精度 FPU，支持 DSP 指令。

项目	STM32F407ZGT6	STM32L496VET6
官方手册	DS8626 Rev 12 STM32F405xx / STM32F407xx	DS11585 Rev 20 STM32L496xx
CPU 最高频率	168 MHz	80 MHz
Flash	1 MB	512 KB
SRAM	192 KB，其中 CCM 为 64 KB；另有 4 KB 备份 SRAM	320 KB，其中 64 KB 带硬件奇偶校验
封装	LQFP144	LQFP100
工作电源	1.8～3.6 V	1.71～3.6 V
主要接口	以太网 MAC、USB OTG FS/HS、CAN、USART、SPI、I²C、FSMC	USB OTG FS、CAN、LPUART、I²C、SPI、QUADSPI、SDMMC
模拟与控制	3 个 12 位 ADC、2 个 DAC 通道、多组 TIM	3 个 12 位 ADC、2 个 DAC 通道、运放、比较器、低功耗定时器
低功耗模式	Sleep、Stop、Standby	Sleep、低功耗运行、低功耗睡眠、Stop 0/1/2、Standby、Shutdown

普中 F407-T100 采用 STM32F407ZGT6，潘多拉板的主芯片为 STM32L475VET6。

8、扩展练习（依据个人选做，建议都做）：

①查阅淘宝上 STM32F407ZGT6 最小系统的电路图，贴出来。

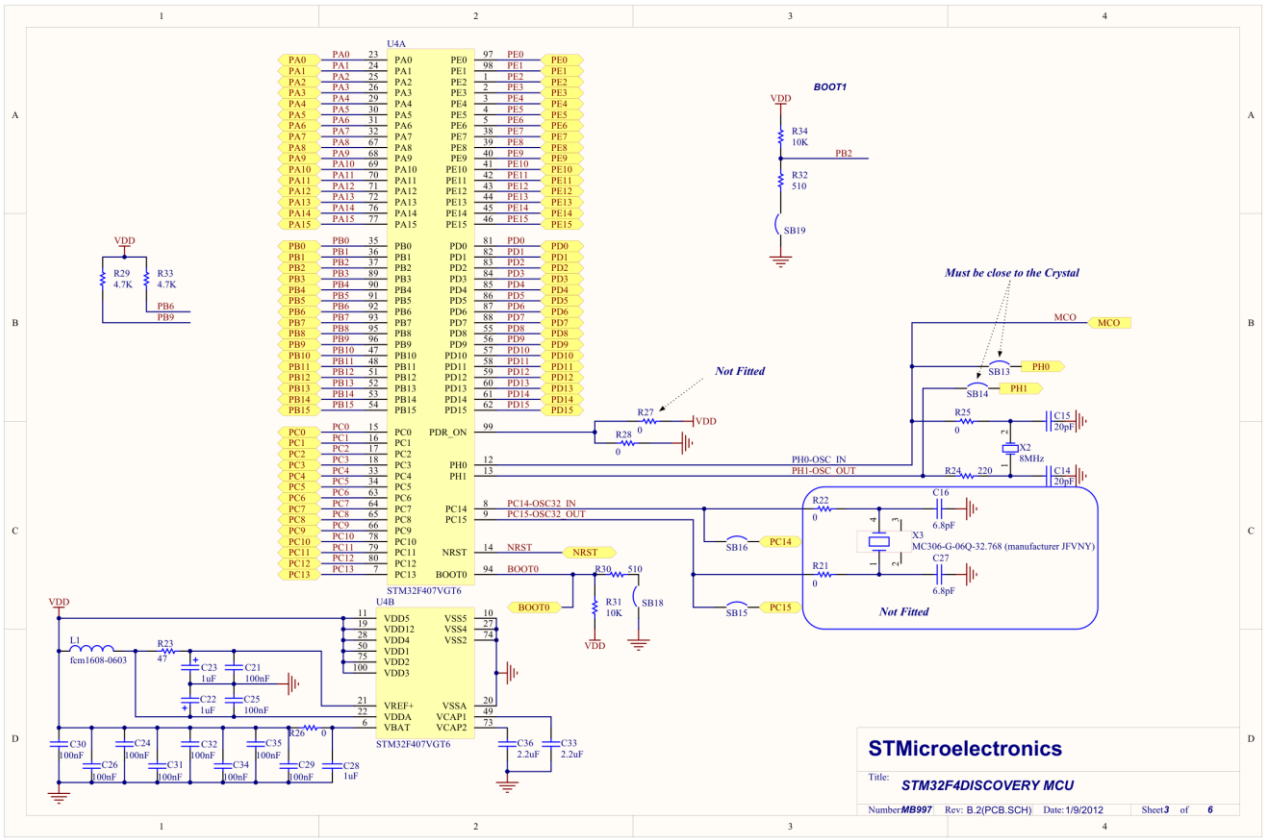


图 6 STM32F407 最小系统电路图

②参考网络教学平台的资源（安装软件和录制视频），构建标准库和抽象库的空白工程，观察 group、main.c 等信息，理解模板的作用。

SPL_GPIO 和 HAL_GPIO 目录各有 Blank（空白模板）与 Template（GPIO 示例）两个 µVision 工程；Group 分组管理源文件，main.c 为应用入口，模板固定了芯片型号、头文件路径和编译配置。

组成	标准库工程	HAL 工程
用户程序	main_blank.c 或 main.c 及中断处理文件	main_blank.c 或 main.c 及中断处理文件
启动与内核	startup_stm32f40_41xxx.s CMSIS 与 system_stm32f4xx.c	startup_stm32f407xx.s CMSIS 与 system_stm32f4xx.c
驱动库	STM32F4xx_StdPeriph_Driver	STM32F4xx_HAL_Driver
空白模板入口	main → while (1)	HAL_Init → SystemClock_Config → while (1)

③打开“startup_stm32f40_41xxx.s”或 startup_stm32f407xx.s 文件，观察有关向量表、堆栈等信息，理解复位序列。

启动文件的向量表共 98 项：16 个系统表项和 82 个外部中断表项；Stack_Size = 0x00000400（1 KB 栈），Heap_Size = 0x00000200（512 B 堆），DCD 按 32 位字放置入口地址。向量表和复位入口节选如下。

```
__Vectors       DCD     __initial_sp
                DCD     Reset_Handler
                DCD     NMI_Handler
                DCD     HardFault_Handler

Reset_Handler   PROC
                EXPORT  Reset_Handler [WEAK]
                IMPORT  SystemInit
                IMPORT  __main
                LDR     R0, =SystemInit
                BLX     R0
                LDR     R0, =__main
                BX      R0
                ENDP
```

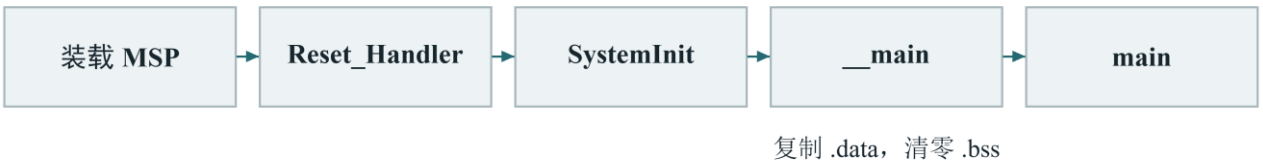


图 7 从复位到 main 的执行顺序

④新建一个 GPIO 的工程，体会标准库和抽象库中初始化、读引脚、写引脚函数的使用。

PF9 接低电平点亮的 LED，PE4 接按下接地的按键，初始化后循环读按键、写 LED。

代码 1 SPL_GPIO/User/main.c 完整内容

```
#include "stm32f4xx.h"

static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio;
    RCC_AHB1PeriphClockCmd(
        RCC_AHB1Periph_GPIOF | RCC_AHB1Periph_GPIOE, ENABLE);

    GPIO_SetBits(GPIOF, GPIO_Pin_9);
    GPIO_StructInit(&gpio);
    gpio.GPIO_Pin = GPIO_Pin_9;
    gpio.GPIO_Mode = GPIO_Mode_OUT;
    gpio.GPIO_OType = GPIO_OType_PP;
    gpio.GPIO_Speed = GPIO_Speed_2MHz;
    gpio.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOF, &gpio);

    gpio.GPIO_Pin = GPIO_Pin_4;
    gpio.GPIO_Mode = GPIO_Mode_IN;
    gpio.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOE, &gpio);
}

int main(void)
{
    GPIO_Setup();
    while (1)
    {
        if (GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4) == Bit_RESET)
            GPIO_ResetBits(GPIOF, GPIO_Pin_9);
        else
            GPIO_SetBits(GPIOF, GPIO_Pin_9);
    }
}
```

④新建一个 GPIO 的工程，体会标准库和抽象库中初始化、读引脚、写引脚函数的使用。

HAL 工程沿用 PF9、PE4 接线，8 MHz HSE 经 PLL 配为 168 MHz。

代码 2 HAL_GPIO/User/main.c 中的初始化与主循环

```
static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio = {0};
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();

    HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, GPIO_PIN_SET);
    gpio.Pin = GPIO_PIN_9;
    gpio.Mode = GPIO_MODE_OUTPUT_PP;
    gpio.Pull = GPIO_NOPULL;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOF, &gpio);

    gpio.Pin = GPIO_PIN_4;
    gpio.Mode = GPIO_MODE_INPUT;
    gpio.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOE, &gpio);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    GPIO_Setup();
    while (1)
    {
        GPIO_PinState key = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_4);
        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, key);
    }
}
```

操作	标准库	HAL
打开时钟	RCC_AHB1PeriphClockCmd	__HAL_RCC_GPIOx_CLK_ENABLE
初始化	GPIO_Init	HAL_GPIO_Init
读输入	GPIO_ReadInputDataBit	HAL_GPIO_ReadPin
写输出	GPIO_SetBits / GPIO_ResetBits	HAL_GPIO_WritePin

编译记录

编译环境为 Keil µVision，编译器为 Arm Compiler 5.06 update 7 build 960。四个工程的编译结果如下。

工程	编译结果	代码与只读数据
SPL_Blank	0 错误，0 警告	Code 704 B，RO-data 408 B
HAL_Blank	0 错误，0 警告	Code 3106 B，RO-data 450 B
SPL_GPIO	0 错误，0 警告	Code 1068 B，RO-data 408 B
HAL_GPIO	0 错误，0 警告	Code 3814 B，RO-data 450 B